Agenda

Angular Getting Started

- Introduction to Angular
- Angular Version History
- Angular CLI
- Initialization Process
- Angular Building Blocks
- Single Page App
- MPA vs. SPA

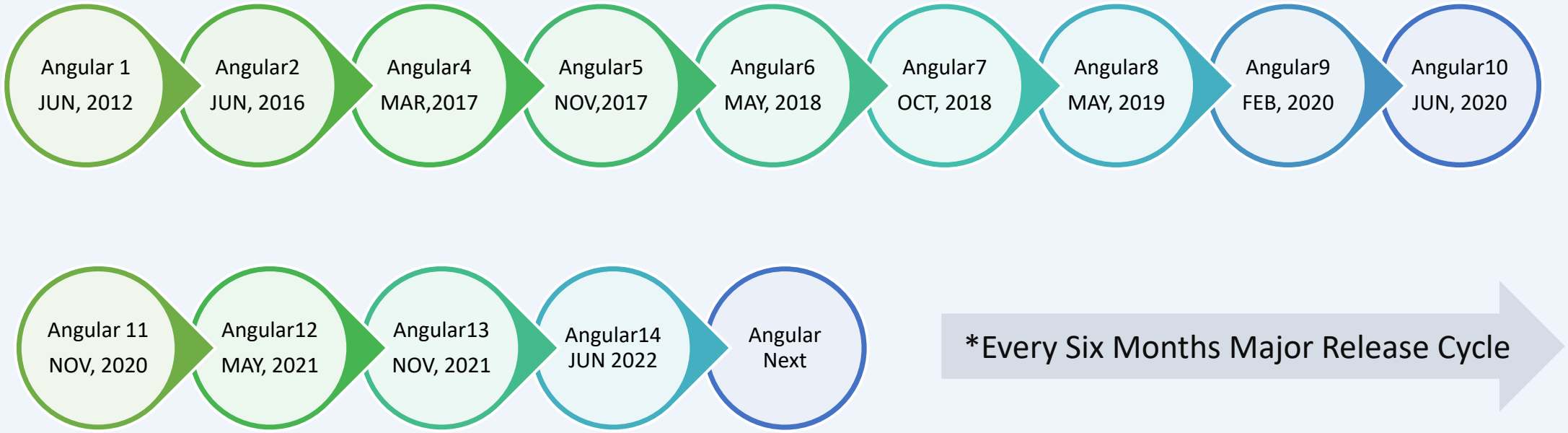
Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Introduction to Angular

- An open-source framework for building single-page applications (SPA) using web technologies like html, css and js.
- Angular is written in TypeScript and follows TypeScript syntax to write code.
- Empowers developers to build applications for browsers, mobiles, or desktop

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Angular Version History



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

ScholarHat

Angular CLI

- A powerful to create, build, compile and serve Angular2 App.
- Used to generate new components, routes, services, and pipes.
- Installing Angular CLI
  - *npm install -g @angular/cli*
- Generating and serving Angular app
  - *ng new proj_name --skip-install*
  - *cd proj_name*
  - *npm install*
  - *ng serve*

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Angular CLI Advantages

- Follow Angular Best Practices
- Configure Style Guides like css, saas
- Use Dev Server Webpack
- Handle Environments
- Build Management
- Testing using Karma and Protractor

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Angular CLI Options

Options	Usage
Help	ng --help
Build	ng build *--env*
Build and Run	ng serve
Testing	ng test
End-End Testing	ng e2e

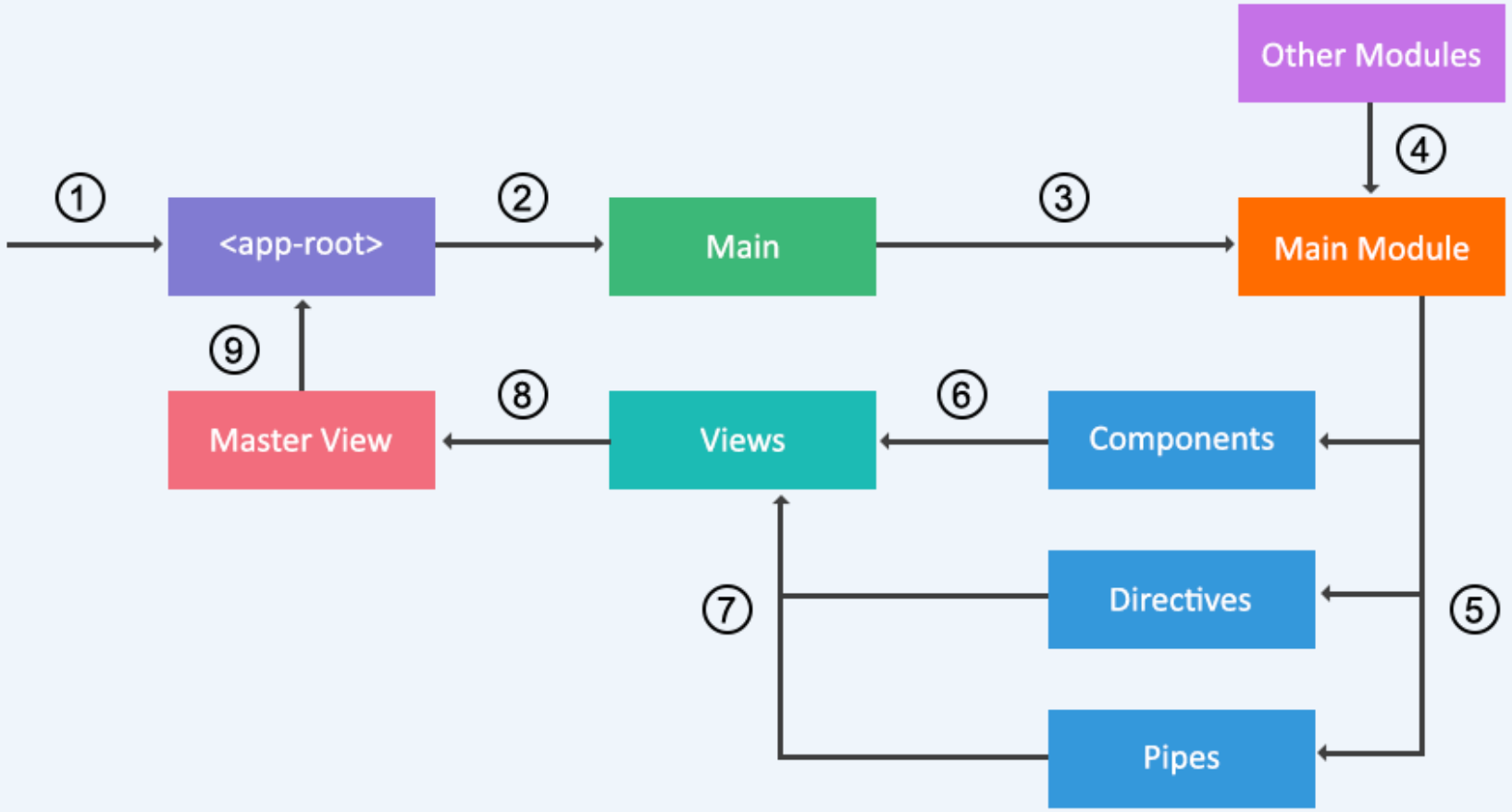
Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Angular CLI Commands

Scaffold	Usage	In Short
Module	ng generate module *my-module*	ng g m *my-module*
Component	ng generate component *my-component*	ng g c *my-component*
Directive	ng generate directive *my-directive*	ng g d *my-directive*
Pipe	ng generate pipe *my-pipe*	ng g p *my-pipe*
Service	ng generate service *my-service*	ng g s *my-service*
Guard	ng generate guard *my-guard*	ng g g *my-guard*
Class	ng generate class *my-class*	ng g cl *my-class*
Interface	ng generate interface *my-interface*	ng g i *my-interface*
Enum	ng generate enum *my-enum*	ng g e *my-enum*

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Angular Initialization Process



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

ScholarHat

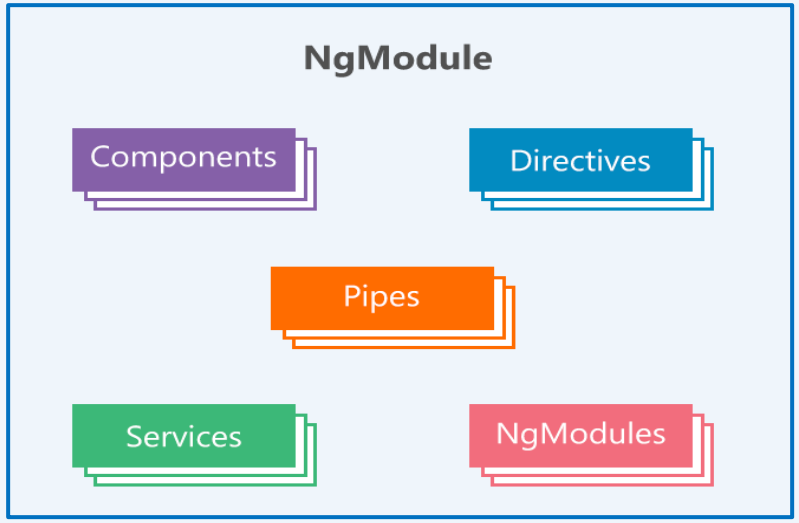
Angular Building Blocks

- Modules
- Components
- Templates
- Metadata
- Data binding
- Directives
- Pipes
- Routing
- Forms
- Services
- Hooks
- Dependency injection

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Modules

- A module organize an application into unified blocks of functionality
- An Angular module is a class with an *@NgModule* decorator
- Accepts a single metadata object whose properties describe the module
- Each Angular app must have at least one module, known as root module



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

NgModule Metadata Main Properties

- **imports** – Specify other dependent modules whose classes are required by the component templates declared in the module
- **declarations** – Specify the components, directives, and pipes that belong to the module
- **bootstrap** – Specify the main app view i.e root component. Only the **root module** can have this bootstrap property
- **exports** – A subset of declarations that will be visible and usable in the other modules. A root module doesn't have export option.
- **providers** – Specify the services, accessible across the app

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Built-In Modules

- Built-In library modules starting with the @angular as the prefix.

- Built-In library & third-party modules can be installed using npm manager.
- Built-In modules, components, services, directives, etc. can be imported by using built-In library modules.

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Component

- A type of directives with template, styles and logic for user interaction
- Exported as a custom HTML tag like as:
  - <my-component></my-component>
- Initialized by Angular Dependency Injection engine

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Component Example

```
import { Component} from '@angular/core';

@Component({
  selector: 'my-component',
  template: `<h3>Interpolation</h3>
            <p>Name : {{name}}</p>
            <p><input type="text" value="{{name}}" /></p>`,
  styles: []
})
export class MyComponent {
  name: string = 'Shailendra';
  constructor() { }
}
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Angular Components Page View

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Template

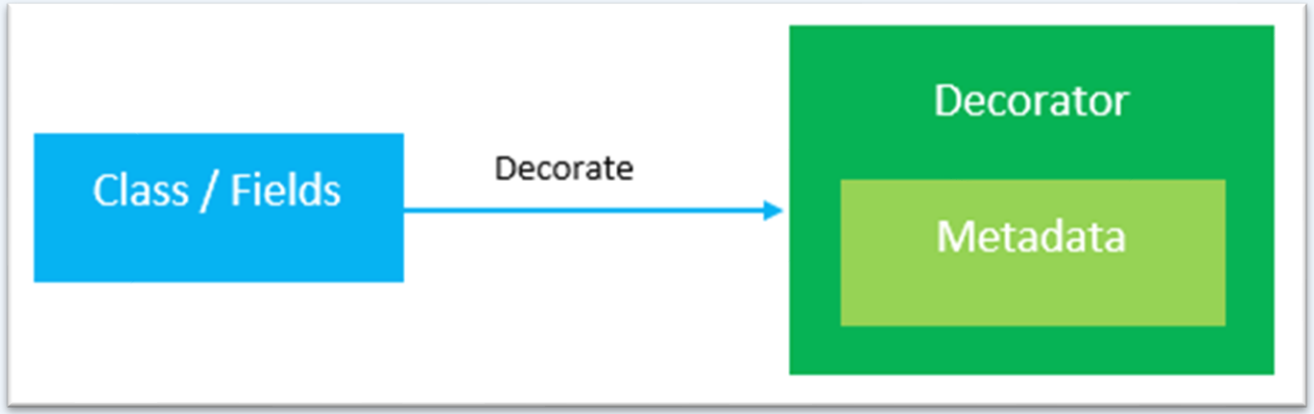
- Define the view of a component.
- Contains Html markup and angular directives, attributes etc.
- Describe how a component is rendered on the page.

```
<h3>Interpolation</h3>
<p>Name : {{name}}</p>
<p>
  <input type="text" value="{{name}}" />
</p>
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Decorators

- A function that adds metadata to a class, class members
- These are prefix with @ symbol
- Angular has built-In decorators like - @Component, @NgModule, @Directive, @Pipe etc.



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Types of Decorators

- Class decorators
  - @NgModule – Used for defining a module
  - @Component – Used for defining a component
  - @Directive – Used for defining a directive
  - @Injectable – Used for injecting dependencies
  - @Pipe – Used for defining a pipe
- Class field decorators
  - @Input – Used for receiving data (input) from parent to child component
  - @Output – Used for passing data (events) from child to parent component

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Metadata

- Tells Angular how to process a class
- Decorators are used to attach metadata to a class

```
@Component({
  selector: 'my-component',
  template: `<p>{{name}}</p>`,
  styles: []
})
export class MyComponent {
  name: string = 'Shailendra Chauhan';
  constructor() { }
}
```

Component Decorators

Directive Name used in HTML

Views

Binding

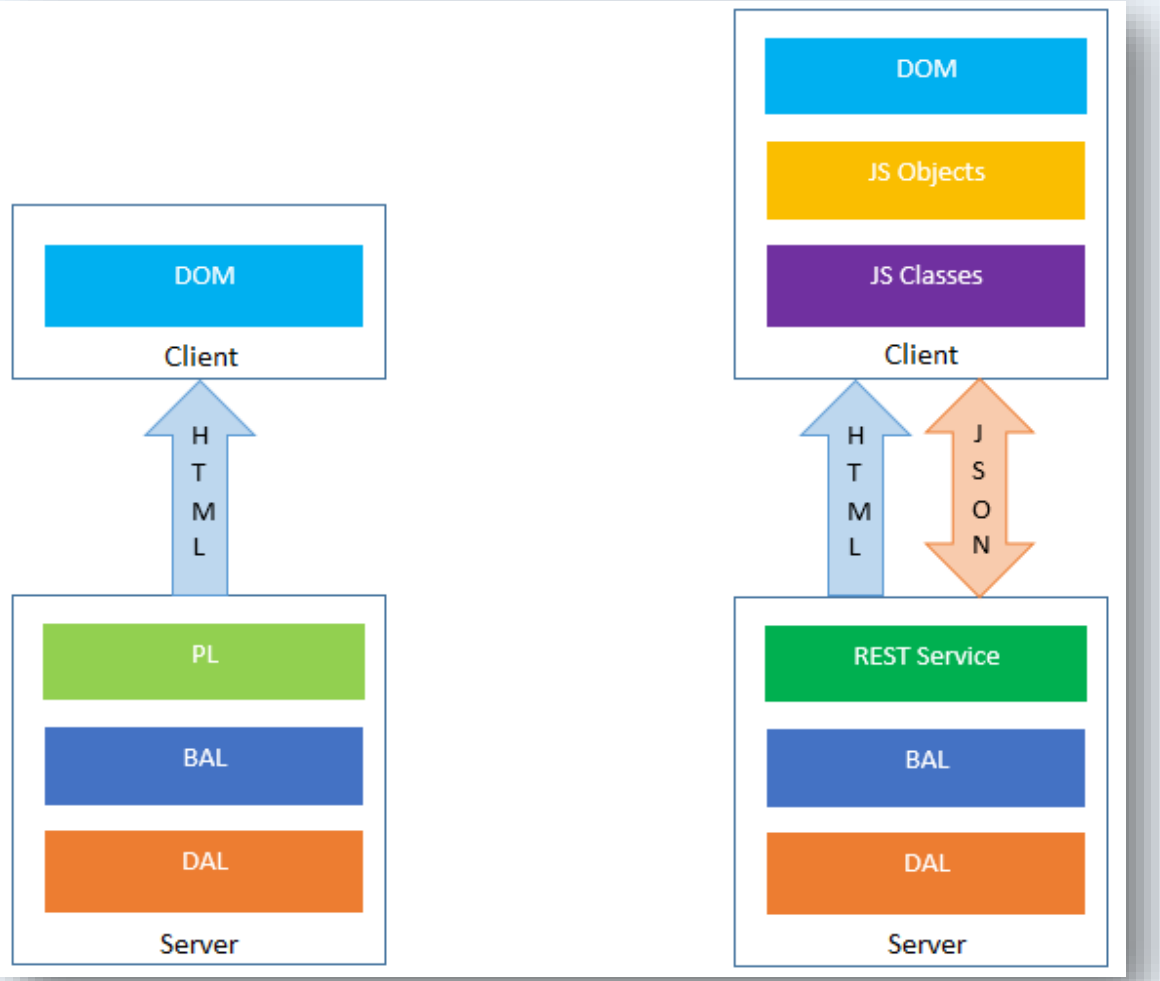
Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Single Page App

- Single HTML page is loaded when the app is loaded.
- Heavy emphasis on JS & UX (User Experience).
- Consumes data asynchronously from a RESTful API.
- Typically, the URL doesn't change except for hash (#).
- No page reloads.
- Universally accessible through a web browser.

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Multiple Page App vs. Single Page App



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

ScholarHat

THANKS

Any Questions?

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

ScholarHat